

Querying Datas- Sort and Joins

Inserted all the data into departments table, location table, employees table.

The screenshot shows the MySQL Workbench interface. On the left, the 'Schemas' tab is active, showing a tree view of databases including 'employee', 'sakila', 'sys', and 'world'. The 'employee' database is selected. The main editor window displays the SQL File 3* with the following code:

```
58 CREATE TABLE departments(  
59     department_id int PRIMARY KEY,  
60     department_name VARCHAR(100) NOT NULL UNIQUE  
61 );  
62 INSERT INTO departments (department_id, department_name) VALUES  
63 (1, 'Software Development'),  
64 (2, 'Marketing'),  
65 (3, 'Data Science'),  
66 (4, 'Human Resources'),  
67 (5, 'Product Management'),  
68 (6, 'Content Creation'),  
69 (7, 'Finance'),  
70 (8, 'Design'),  
71 (9, 'Research and Development'),  
72 (10, 'Customer Support'),  
73 (11, 'Business Development'),  
74 (12, 'IT'),  
75 (13, 'Operations');  
76
```

The Output window at the bottom shows the execution results:

#	Time	Action	Message
1	22:03:16	USE employee	0 row(s) affected
2	22:03:20	CREATE TABLE departments(department_id int PRIMARY KEY, department_name VARCHAR...	0 row(s) affected
3	22:03:25	INSERT INTO departments (department_id, department_name) VALUES (1, 'Software Develop...	13 row(s) affected Records: 13 Duplicates: 0 Warnings: 0

The screenshot shows the MySQL Workbench interface with the SQL File 4* window. The code in the editor is as follows:

```
22 (13, 'Operations');  
23  
24 CREATE TABLE Location(  
25     location_id INT PRIMARY KEY AUTO_INCREMENT,  
26     location VARCHAR(30) NOT NULL UNIQUE  
27 );  
28 INSERT INTO Location (location) VALUES  
29 ('Chennai'),  
30 ('Bangalore'),  
31 ('Hyderabad'),  
32 ('Pune');  
33  
34 CREATE TABLE employees(  
35     employee_id INT PRIMARY KEY,  
36     employee_name VARCHAR(50) NOT NULL,  
37     gender enum('M', 'F'),  
38     age INT CHECK(age>=18),  
39     hire_date DATE DEFAULT(current_date),  
40     designation VARCHAR(100),
```

The Output window at the bottom shows the execution results:

#	Time	Action	Message
4	22:10:50	INSERT INTO departments (department_id, department_name) VALUES (1, 'Software Develo...	13 row(s) affected Records: 13 Duplicates: 0 Warnings: 0
5	22:10:56	CREATE TABLE Location(location_id INT PRIMARY KEY AUTO_INCREMENT, location ...	0 row(s) affected
6	22:11:01	INSERT INTO Location (location) VALUES ('Chennai'), ('Bangalore'), ('Hyderabad'), ('Pune')	4 row(s) affected Records: 4 Duplicates: 0 Warnings: 0

Navigator: MySQL-employeeDB-ddl-constr... SQL File 3* SQL File 4* x

Limit to 1000 rows

```

42 location_id INT,
43 salary DECIMAL(10,2),
44 FOREIGN KEY (department_id) REFERENCES Departments (department_id),
45 FOREIGN KEY (location_id) REFERENCES Location (location_id)
46 );
47 • INSERT INTO employees (employee_id, employee_name, gender, age, hire_date, designation, department_id, location_id, salary)
48 (5001, 'Vihaan Singh', 'M', 27, '2015-01-20', 'Data Analyst', 3, 4, 60000),
49 (5002, 'Reyansh Singh', 'M', 31, '2015-03-10', 'Network Engineer', 12, 1, 80000),
50 (5003, 'Aaradhyia Iyer', 'F', 26, '2015-05-20', 'Customer Support Executive', 10, 2, 45000),
51 (5004, 'Kiara Malhotra', 'F', 29, '2015-07-05', NULL, 8, 3, 70000),
52 (5005, 'Anvi Chaudhary', 'F', 25, '2015-09-11', 'Business Development Executive', 11, 1, 55000),
53 (5006, 'Dhruv Shetty', 'M', 28, '2015-11-20', 'UI Developer', 8, 2, 65000),
54 (5007, 'Anushka Singh', 'F', 32, '2016-01-15', 'Marketing Manager', 2, 3, 90000),
55 (5008, 'Diya Jha', 'F', 27, '2016-03-05', 'Graphic Designer', 8, 4, 70000),
56 (5009, 'Kiaan Desai', 'M', 30, '2016-05-20', 'Sales Executive', 11, 3, 55000),
57 (5010, 'Atharv Yadav', 'M', 29, '2016-07-10', 'Systems Administrator', 12, 4, 80000),
58 (5011, 'Saanvi Patel', 'F', 28, '2016-09-20', 'Marketing Analyst', 2, 1, 60000),
59 (5012, 'Myra Verma', 'F', 26, '2016-11-05', 'Operations Manager', 13, 2, 95000),
60 (5013, 'Arnav Rao', 'M', 33, '2017-01-20', 'Customer Success Manager', 10, 3, 75000),
61 (5014, 'Vihaan Mohan', 'M', 30, '2017-03-10', 'Supply Chain Analyst', 10, 2, 60000).

```

Output: Action Output

#	Time	Action	Message
8	22:11:36	CREATE TABLE employees(employee_id INT PRIMARY KEY, employee_name VARCHAR...	0 row(s) affected
9	22:11:46	INSERT INTO employees (employee_id, employee_name, gender, age, hire_date, designation...	30 row(s) affected Records: 30 Duplicates: 0 Warnings: 0

1. Distinct Values:

- A query to retrieve distinct salaries from the Employees table.

SELECT DISTINCT salary FROM employees ;

Navigator: MySQL-employeeDB-ddl-constr... SQL File 3* SQL File 4* x

Limit to 1000 rows

```

68 (5021, 'Advik Kapoor', 'M', 26, '2018-05-20', 'Finance Analyst', 7, 3, 85000),
69 (5022, 'Aadhya Iyengar', 'F', 28, '2018-07-10', 'HR Specialist', 4, 4, 60000),
70 (5023, 'Anika Paul', 'F', 30, '2018-09-20', 'Public Relations Specialist', 2, 2, 70000),
71 (5024, 'Aryan Shetty', 'M', 27, '2018-11-05', 'Product Manager', 5, 1, 95000),
72 (5025, 'Avni Iyengar', 'F', 31, '2019-01-20', 'Data Scientist', 3, 4, 100000),
73 (5026, 'Vivaan Singh', 'M', 29, '2019-03-10', 'Business Analyst', 3, 2, 75000),
74 (5027, 'Ananya Paul', 'F', 32, '2019-05-20', 'Content Writer', 6, 3, 60000),
75 (5028, 'Anaya Kapoor', 'F', 26, '2019-07-05', 'Event Coordinator', 6, 1, 60000),
76 (5029, 'Arjun Kumar', 'M', 33, '2019-09-11', 'Quality Assurance Analyst', 12, 2, 80000),
77 (5030, 'Sara Iyer', 'F', 28, '2019-11-20', 'Project Manager', 5, 1, 90000);
78
79 • SELECT DISTINCT salary FROM employees ;

```

Result Grid: Filter Rows: Export: Wrap Cell Content: 1A

salary
60000.00
80000.00
45000.00
70000.00
55000.00
65000.00
90000.00
95000.00
75000.00
85000.00
100000.00

employees 1 x

2. Alias (AS):

- Provide aliases for the "age" and "salary" columns as "Employee_Age" and "Employee_Salary", respectively.

SELECT age AS Employee_Age, salary AS Employee_Salary FROM employees;

Navigator: MySQL-employeeDB-ddl-constr... SQL File 3* SQL File 4* x

CHEMAS

Filter objects

- employee_db
- sakila
- sys
- world

administration Schemas

information

No object selected

```

72 (5025, 'Avni Iyengar', 'F', 31, '2019-01-20', 'Data Scientist', 3, 4, 100000),
73 (5026, 'Vivaan Singh', 'M', 29, '2019-03-10', 'Business Analyst', 3, 2, 75000),
74 (5027, 'Ananya Paul', 'F', 32, '2019-05-20', 'Content Writer', 6, 3, 60000),
75 (5028, 'Anaya Kapoor', 'F', 26, '2019-07-05', 'Event Coordinator', 6, 1, 60000),
76 (5029, 'Arjun Kumar', 'M', 33, '2019-09-11', 'Quality Assurance Analyst', 12, 2, 80000),
77 (5030, 'Sara Iyer', 'F', 28, '2019-11-20', 'Project Manager', 5, 1, 90000);
78
79 -- A query to retrieve distinct salaries from the Employees table.
80 • SELECT DISTINCT salary FROM employees ;
81
82 -- Provide aliases for the "age" and "salary" columns as "Employee_Age" and "Employee_Salary", respectively.
83 • SELECT age AS Employee_Age,salary AS Employee_Salary FROM employees;

```

Result Grid

Employee_Age	Employee_Salary
27	60000.00
31	80000.00
26	45000.00
29	70000.00
25	55000.00
28	65000.00
32	90000.00
27	70000.00
30	55000.00
29	80000.00
28	60000.00

employees 2 x

3. Where Clause & Operators:

● Retrieve employees with a salary greater than ₹50000 and hired before 2016-01-01.

SELECT employee_id,employee_name, salary FROM employees

WHERE salary>50000 AND hire_date <'2016-01-01';

Navigator: MySQL-employeeDB-ddl-constr... SQL File 3* SQL File 4* x

SCHEMAS

Filter objects

- employee_db
- sakila
- sys
- world

Administration Schemas

Information

No object selected

```

79 -- A query to retrieve distinct salaries from the Employees table.
80 • SELECT DISTINCT salary FROM employees ;
81
82 -- Provide aliases for the "age" and "salary" columns as "Employee_Age" and "Employee_Salary", respectively.
83 • SELECT age AS Employee_Age,salary AS Employee_Salary FROM employees;
84
85 -- Retrieve employees with a salary greater than ₹50000 and hired before 2016-01-01.
86 • SELECT employee_id,employee_name, salary
87 FROM employees
88 WHERE salary>50000 AND hire_date <'2016-01-01';

```

Result Grid

employee_id	employee_name	salary
5001	Vihaan Singh	60000.00
5002	Reyansh Singh	80000.00
5004	Kiara Malhotra	70000.00
5005	Anvi Chaudhary	55000.00
5006	Dhruv Shetty	65000.00

employees 4 x

Output

Action Output

#	Time	Action	Message
13	22:29:42	SELECT employee_id,employee_name, salary FROM employees WHERE salary>50000 AN...	0 row(s) returned
14	22:31:34	SELECT employee_id,employee_name, salary FROM employees WHERE salary>50000 AN...	5 row(s) returned

Object Info Session

🕒 Find the employee whose designation is missing and fill it with "DataScientist".

```
SELECT * FROM employees
```

```
WHERE designation IS NULL OR designation=' ';
```

```
UPDATE employees SET designation="Data Scientist"
```

```
WHERE employee_id=5004;
```

```
SELECT * FROM employees;
```

The screenshot shows the MySQL Workbench interface. The left sidebar displays the 'SCHEMAS' tree with 'employee_db' selected. The main editor window shows a SQL script with the following queries:

```
-- A query to retrieve distinct salaries from the Employees table.
SELECT DISTINCT salary FROM employees ;

-- Provide aliases for the "age" and "salary" columns as "Employee_Age" and "Employee_Salary", respectively.
SELECT age AS Employee_Age,salary AS Employee_Salary FROM employees;

-- Retrieve employees with a salary greater than ₹50000 and hired before 2016-01-01.
SELECT employee_id,employee_name, salary
FROM employees
WHERE salary>50000 AND hire_date <'2016-01-01';

-- Find the employee whose designation is missing and fill it with "DataScientist".
SELECT * FROM employees
WHERE designation IS NULL OR designation=' ';
```

The 'Result Grid' at the bottom shows the results of the last query:

employee_id	employee_name	gender	age	hire_date	designation	department_id	location_id	salary
5004	Kiara Malhotra	F	29	2015-07-05	NULL	8	3	70000.00

The screenshot shows the MySQL Workbench interface after executing the update query. The SQL script now includes:

```
-- Retrieve employees with a salary greater than ₹50000 and hired before 2016-01-01.
SELECT employee_id,employee_name, salary
FROM employees
WHERE salary>50000 AND hire_date <'2016-01-01';

-- Find the employee whose designation is missing and fill it with "DataScientist".
SELECT * FROM employees
WHERE designation IS NULL OR designation=' ';
```

The 'Result Grid' shows the updated data:

employee_id	employee_name	gender	age	hire_date	designation	department_id	location_id	salary
5001	Vihaan Singh	M	27	2015-01-20	Data Analyst	3	4	60000.00
5002	Reyansh Singh	M	31	2015-03-10	Network Engineer	12	1	80000.00
5003	Aaradhyia Iyer	F	26	2015-05-20	Customer Support Executive	10	2	45000.00
5004	Kiara Malhotra	F	29	2015-07-05	Data Scientist	8	3	70000.00
5005	Anvi Chaudhary	F	25	2015-09-11	Business Development Executive	11	1	55000.00

The 'Output' tab at the bottom shows the execution log:

#	Time	Action	Message
22	22:42:55	UPDATE employees SET designation="Data Scientist" WHERE employee_id=5004	1 row(s) affected Rows matched: 1 Changed: 1 Warnings: 0
23	22:43:31	SELECT * FROM employees LIMIT 0, 1000	30 row(s) returned

Sorting and Grouping Data:

1. ORDER BY:

- Find employees sorted by department ID in ascending order and salary in descending order.

```
SELECT * FROM employees
```

```
ORDER BY department_id ASC, salary DESC;
```

The screenshot shows the MySQL Workbench interface. The SQL Editor contains the following query:

```
92 WHERE designation IS NULL OR designation='';
93
94 UPDATE employees SET designation="Data Scientist"
95 WHERE employee_id=5004;
96 SELECT * FROM employees;
97
98 -- Find employees sorted by department ID in ascending order and salary in descending order.
99 SELECT * FROM employees
100 ORDER BY department_id ASC, salary DESC;
101
```

The Result Grid displays the following data:

employee_id	employee_name	gender	age	hire_date	designation	department_id	location_id	salary
5020	Aarav Nair	M	32	2018-03-05	Software Engineer	1	1	90000.00
5007	Anushka Singh	F	32	2016-01-15	Marketing Manager	2	3	90000.00
5023	Anika Paul	F	30	2018-09-20	Public Relations Specialist	2	2	70000.00
5011	Saanvi Patel	F	28	2016-09-20	Marketing Analyst	2	1	60000.00
5025	Avni Iyengar	F	31	2019-01-20	Data Scientist	3	4	100000.00
5026	Vivaan Singh	M	29	2019-03-10	Business Analyst	3	2	75000.00
5001	Vhaan Singh	M	27	2015-01-20	Data Analyst	3	4	60000.00
5016	Zoya Khan	F	31	2017-07-05	Legal Counsel	4	4	100000.00
5022	Aadhya Iyengar	F	28	2018-07-10	HR Specialist	4	4	60000.00
5019	Ishika Patel	F	29	2018-01-15	Talent Acquisition Specialist	4	4	55000.00
5024	Aryan Shetty	M	27	2018-11-05	Product Manager	5	1	95000.00
5030	Sara Iyer	F	28	2019-11-20	Project Manager	5	1	90000.00

2. LIMIT:

- Display the first 5 employees hired in the year 2018.

```
SELECT * FROM employees WHERE YEAR(hire_date)='2018' LIMIT 5;
```

The screenshot shows the MySQL Workbench interface. The SQL Editor contains the following query:

```
92 WHERE designation IS NULL OR designation='';
93
94 UPDATE employees SET designation="Data Scientist"
95 WHERE employee_id=5004;
96 SELECT * FROM employees;
97
98 -- Find employees sorted by department ID in ascending order and salary in descending order.
99 SELECT * FROM employees
100 ORDER BY department_id ASC, salary DESC;
101
102 -- Display the first 5 employees hired in the year 2018.
103 SELECT * FROM employees
104 WHERE YEAR(hire_date)='2018'
105 LIMIT 5;
```

The Result Grid displays the following data:

employee_id	employee_name	gender	age	hire_date	designation	department_id	location_id	salary
5019	Ishika Patel	F	29	2018-01-15	Talent Acquisition Specialist	4	4	55000.00
5020	Aarav Nair	M	32	2018-03-05	Software Engineer	1	1	90000.00
5021	Advik Kapoor	M	26	2018-05-20	Finance Analyst	7	3	85000.00
5022	Aadhya Iyengar	F	28	2018-07-10	HR Specialist	4	4	60000.00
5023	Anika Paul	F	30	2018-09-20	Public Relations Specialist	2	2	70000.00

3. Aggregate Functions:

- Calculate the sum of all salaries in the Finance department.

```
SELECT * FROM departments WHERE department_name="Finance";
```

```
SELECT SUM(salary) FROM employees
```

```
WHERE department_id=7;
```

The screenshot shows the MySQL Workbench interface. The SQL Editor contains the following queries:

```
101
102 -- Display the first 5 employees hired in the year 2018.
103 • SELECT * FROM employees
104 WHERE YEAR(hire_date)='2018'
105 LIMIT 5;
106
107 -- Calculate the sum of all salaries in the Finance department
108 • SELECT * FROM departments WHERE department_name="Finance";
109 • SELECT SUM(salary) FROM employees
110 WHERE department_id=7;
111
```

The Results pane shows the output of the first query (line 108):

department_name
Finance

The Results pane also shows the output of the second query (line 109):

SUM(salary)
170000.00

The Output pane shows the execution log:

#	Time	Action	Message
7	20:00:05	SELECT * FROM departments WHERE department_name="Finance" LIMIT 0, 1000	1 row(s) returned
8	20:00:05	SELECT SUM(salary) FROM employees WHERE department_id=7 LIMIT 0, 1000	1 row(s) returned

- Find the minimum age among all employees.

```
SELECT MIN(age) FROM employees;
```

The screenshot shows the MySQL Workbench interface. The SQL Editor contains the following queries:

```
103 • SELECT * FROM employees
104 WHERE YEAR(hire_date)='2018'
105 LIMIT 5;
106
107 -- Calculate the sum of all salaries in the Finance department
108 • SELECT * FROM departments WHERE department_name="Finance";
109 • SELECT SUM(salary) FROM employees
110 WHERE department_id=7;
111
112 -- Find the minimum age among all employees.
113 • SELECT MIN(age) FROM employees;
```

The Results pane shows the output of the first query (line 108):

department_name
Finance

The Results pane also shows the output of the second query (line 109):

SUM(salary)
170000.00

The Results pane also shows the output of the third query (line 113):

MIN(age)
25

The Output pane shows the execution log:

#	Time	Action	Message
7	20:00:05	SELECT * FROM departments WHERE department_name="Finance" LIMIT 0, 1000	1 row(s) returned
8	20:00:05	SELECT SUM(salary) FROM employees WHERE department_id=7 LIMIT 0, 1000	1 row(s) returned
9	20:00:05	SELECT MIN(age) FROM employees;	1 row(s) returned

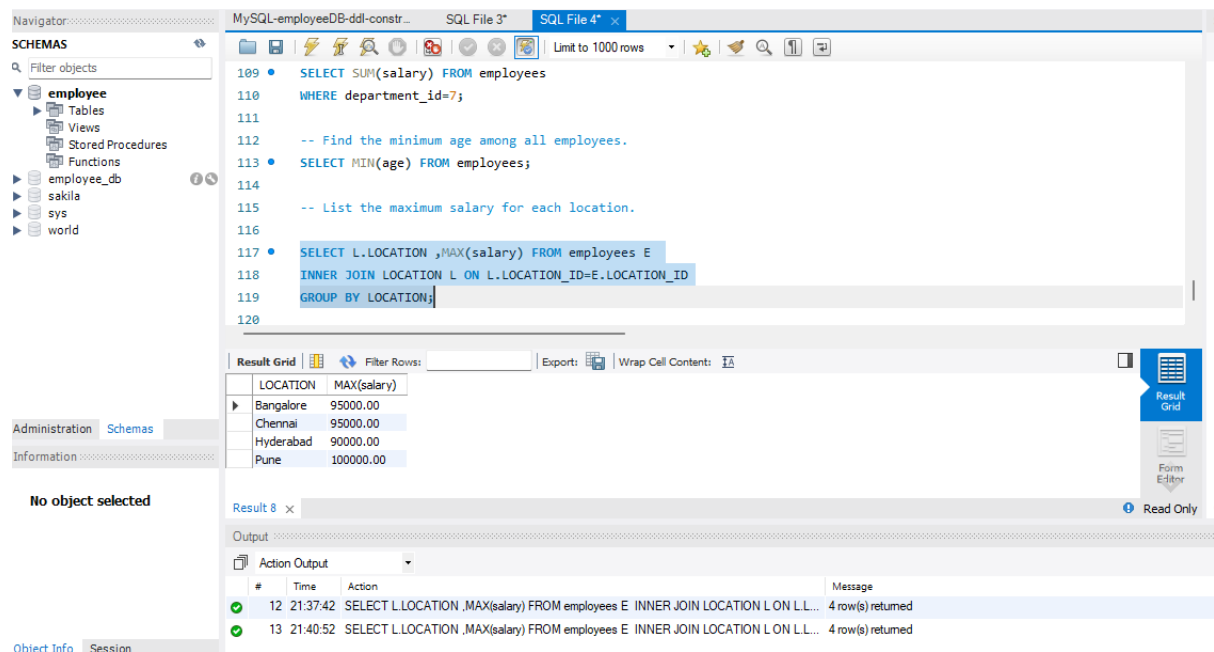
4. GROUP BY:

- List the maximum salary for each location.

```
SELECT L.Location ,MAX(salary) FROM employees E
```

```
INNER JOIN LOCATION L ON L.LOCATION_ID=E.LOCATION_ID
```

```
GROUP BY LOCATION;
```



The screenshot shows a MySQL IDE interface. The left sidebar displays a 'SCHEMAS' tree with 'employee' selected. The main editor shows a SQL script with the following queries:

```
109 • SELECT SUM(salary) FROM employees
110 WHERE department_id=7;
111
112 -- Find the minimum age among all employees.
113 • SELECT MIN(age) FROM employees;
114
115 -- List the maximum salary for each location.
116
117 • SELECT L.LOCATION ,MAX(salary) FROM employees E
118 INNER JOIN LOCATION L ON L.LOCATION_ID=E.LOCATION_ID
119 GROUP BY LOCATION;
120
```

The 'Result Grid' shows the output of the last query:

LOCATION	MAX(salary)
Bangalore	95000.00
Chennai	95000.00
Hyderabad	90000.00
Pune	100000.00

The 'Output' pane shows the execution log:

#	Time	Action	Message
12	21:37:42	SELECT L.LOCATION ,MAX(salary) FROM employees E INNER JOIN LOCATION L ON L.L...	4 row(s) returned
13	21:40:52	SELECT L.LOCATION ,MAX(salary) FROM employees E INNER JOIN LOCATION L ON L.L...	4 row(s) returned

- Calculate the average salary for each designation containing the word

'Analyst'.

```
SELECT designation, AVG(salary) AS Average_Salary
```

```
FROM employees
```

```
WHERE designation LIKE '%Analyst%'
```

```
GROUP BY designation;
```

Navigator: MySQL-employeeDB-ddl-constr... SQL File 3* SQL File 4* x

CHEMAS

Filter objects

- employee
 - Tables
 - Views
 - Stored Procedures
 - Functions
- employee_db
- sakila
- sys
- world

administration Schemas

Information

No object selected

```

116
117 • SELECT L.LOCATION ,MAX(salary) FROM employees E
118 INNER JOIN LOCATION L ON L.LOCATION_ID=E.LOCATION_ID
119 GROUP BY LOCATION;
120
121 -- Calculate the average salary for each designation containing the word 'Analyst'.
122
123 • SELECT designation, AVG(salary) AS Average_Salary
124 FROM employees
125 WHERE designation LIKE '%Analyst%'
126 GROUP BY designation;

```

Result Grid

designation	Average_Salary
Data Analyst	60000.000000
Marketing Analyst	60000.000000
Supply Chain Analyst	60000.000000
Financial Analyst	85000.000000
Finance Analyst	85000.000000
Business Analyst	75000.000000

Result 15 x

Output

Action Output

#	Time	Action	Message
19	21:50:44	SELECT designation,avg(salary) AS Average_Salary FROM employees WHERE Designatio...	0 row(s) returned
20	21:51:37	SELECT designation, AVG(salary) AS Average_Salary FROM employees WHERE designatio...	7 row(s) returned

5. HAVING:

- Find departments with less than 3 employees.

SELECT D.department_name, COUNT(E.employee_id) AS employee_count

FROM employees E

INNER JOIN departments D

ON E.department_id = D.department_id

GROUP BY D.department_name

HAVING COUNT(E.employee_id) < 3;

Navigator: MySQL-employeeDB-ddl-constr... SQL File 3* SQL File 4* x

CHEMAS

Filter objects

- employee
 - Tables
 - Views
 - Stored Procedures
 - Functions
- employee_db
- sakila
- sys
- world

Administration Schemas

Information

No object selected

```

125 WHERE designation LIKE '%Analyst%'
126 GROUP BY designation;
127
128 -- Find departments with less than 3 employees.
129
130 • SELECT D.department_name, COUNT(E.employee_id) AS employee_count
131 FROM employees E
132 INNER JOIN departments D
133 ON E.department_id = D.department_id
134 GROUP BY D.department_name
135 HAVING COUNT(E.employee_id) < 3;
136

```

Result Grid

department_name	employee_count
Business Development	2
Content Creation	2
Finance	2
Operations	1
Product Management	2
Research and Development	1

Result 16 x

Output

Action Output

#	Time	Action	Message
20	21:51:37	SELECT designation, AVG(salary) AS Average_Salary FROM employees WHERE designatio...	7 row(s) returned
21	21:55:44	SELECT D.department_name, COUNT(E.employee_id) AS employee_count FROM employee...	7 row(s) returned

- Find locations with female employees whose average age is below 30.

```
SELECT L.location, AVG(E.age) AS average_age
```

```
FROM employees E
```

```
INNER JOIN location L
```

```
ON L.location_id = E.location_id
```

```
WHERE E.gender = 'F'
```

```
GROUP BY L.location
```

```
HAVING AVG(E.age) < 30;
```

The screenshot shows a MySQL IDE interface. The left sidebar displays a 'SCHEMAS' tree with 'employee' selected. The main editor window shows two SQL queries. The first query is a comment: '-- Find departments with less than 3 employees.' The second query is:

```
SELECT D.department_name, COUNT(E.employee_id) AS employee_count
FROM employees E
INNER JOIN departments D
ON E.department_id = D.department_id
GROUP BY D.department_name
HAVING COUNT(E.employee_id) < 3;
```

 The third query is:

```
-- Find locations with female employees whose average age is below 30.
SELECT L.location, AVG(E.age) AS average_age
FROM employees E
```

 The bottom panel shows the 'Result Grid' for the third query, displaying the following data:

location	average_age
Bangalore	27.3333
Chennai	26.7500
Pune	29.2000

Joins:

1. Inner Join:

- List employee names, their designations, and department names where employees are assigned to a department.

```
SELECT E.EMPLOYEE_NAME,E.DESIGNATION,D.DEPARTMENT_NAME FROM EMPLOYEES E
```

```
INNER JOIN DEPARTMENTS D
```

```
ON D.DEPARTMENT_ID=E.DEPARTMENT_ID ;
```

Navigator: MySQL-employeeDB-ddl-constr... SQL File 3* SQL File 4* x

SCHEMAS

Filter objects

employee

Tables

Views

Stored Procedures

Functions

employee_db

sakila

sys

world

Administration Schemas

Information

No object selected

```

140 INNER JOIN location L
141 ON L.location_id = E.location_id
142 WHERE E.gender = 'F';
143 GROUP BY L.location
144 HAVING AVG(E.age) < 30;
145
146 -- List employee names, their designations, and department names where employees are assigned to a department.
147
148 • SELECT E.EMPLOYEE_NAME,E.DESIGNATION,D.DEPARTMENT_NAME FROM EMPLOYEES E
149 INNER JOIN DEPARTMENTS D
150 ON D.DEPARTMENT_ID=E.DEPARTMENT_ID ;
151

```

Result Grid

EMPLOYEE_NAME	DESIGNATION	DEPARTMENT_NAME
Anvi Chaudhary	Business Development Executive	Business Development
Kiaan Desai	Sales Executive	Business Development
Ananya Paul	Content Writer	Content Creation
Anaya Kapoor	Event Coordinator	Content Creation
Aaradhya Iyer	Customer Support Executive	Customer Support
Ammau Dan	Customer Success Manager	Customer Support

Result 20 x

Output

Action Output

#	Time	Action	Message
28	22:09:29	SELECT E.EMPLOYEE_NAME,E.DESIGNATION,D.DEPARTMENT_NAME FROM EMPLO...	30 row(s) returned

2. Left Join:

- ☉ List all departments along with the total number of employees in each department, including departments with no employees.

```

SELECT D.DEPARTMENT_NAME,
       COUNT(E.EMPLOYEE_ID) AS TOTAL_EMPLOYEES
FROM DEPARTMENTS D
LEFT JOIN EMPLOYEES E
      ON D.DEPARTMENT_ID = E.DEPARTMENT_ID
GROUP BY D.DEPARTMENT_NAME;

```

Navigator: MySQL-employeeDB-ddl-constr... SQL File 3* SQL File 4* x

SCHEMAS

Filter objects

employee

Tables

Views

Stored Procedures

Functions

employee_db

sakila

sys

world

Administration Schemas

Information

No object selected

```

149 INNER JOIN DEPARTMENTS D
150 ON D.DEPARTMENT_ID=E.DEPARTMENT_ID ;
151
152 -- List all departments along with the total number of employees in each department, including departments with no emp
153
154 • SELECT D.DEPARTMENT_NAME,
155        COUNT(E.EMPLOYEE_ID) AS TOTAL_EMPLOYEES
156 FROM DEPARTMENTS D
157 LEFT JOIN EMPLOYEES E
158      ON D.DEPARTMENT_ID = E.DEPARTMENT_ID
159 GROUP BY D.DEPARTMENT_NAME;
160

```

Result Grid

DEPARTMENT_NAME	TOTAL_EMPLOYEES
Business Development	2
Content Creation	2
Customer Support	3
Data Science	3
Design	3
Finance	2

Result 23 x

Output

Action Output

#	Time	Action	Message
36	22:17:21	SELECT D.DEPARTMENT_NAME, COUNT(E.EMPLOYEE_ID) AS TOTAL_EMPLOYE...	13 row(s) returned

3. Right Join:

● Display all locations along with the names of employees assigned to each location. If no employees are assigned to a location, display NULL for employee name.

```
SELECT E.employee_name,L.location FROM Employees E
```

```
RIGHT JOIN Location L
```

```
ON L.location_id=E.location_id;
```

The screenshot shows a MySQL IDE interface. On the left, the 'SCHEMAS' pane displays a tree view with 'employee' selected, showing sub-items like 'Tables', 'Views', 'Stored Procedures', and 'Functions'. Below this, 'Administration' and 'Schemas' tabs are visible, with 'Schemas' active. The main editor window shows a SQL script with the following content:

```
156 FROM DEPARTMENTS D
157 LEFT JOIN EMPLOYEES E
158 ON D.DEPARTMENT_ID = E.DEPARTMENT_ID
159 GROUP BY D.DEPARTMENT_NAME;
160
161 -- Display all locations along with the names of employees assigned to each
162 -- location, display NULL for employee name.
163
164 • SELECT E.employee_name,L.location FROM Employees E
165 RIGHT JOIN Location L
166 ON L.location_id=E.location_id;
```

Below the script, the 'Result Grid' shows the output of the query. It has two columns: 'employee_name' and 'location'. The data is as follows:

employee_name	location
Atharv Yadav	Pune
Zoya Khan	Pune
Ishika Patel	Pune
Aadhya Iyengar	Pune
Avni Iyengar	Pune

At the bottom, the 'Output' pane shows the execution log. The last entry is:

```
38 22:20:42 SELECT E.employee_name,L.location FROM Employees E RIGHT JOIN Location L ON L.lo... 30 row(s) returned
```